Arquitectura general del sistema (tres capas)

Prototipo académico — ningún componente está certificado como dispositivo médico

1. Dispositivo de adquisición (ESP32-S3)

Sensores de presión

- 12 × FSR (1–10 kg fábrica)
- Divisor de tensión + R fija
- Distribuidos en la plantilla

Sensores de temperatura

- 4 × NTC (termorresistencia)
- Steinhart-Hart simplificada
- Distribuidos en la plantilla

IMU LSM9DS1

- 9 grados de libertad
- Accel + gyro + magnetómetro
- Bus I2C 0 (junto a ESP32-S3)

Alimentación

- Batería + regulador
- esp_pm_configure 80–160MHz
- light-sleep / modem-sleep

4 × ADS1115 (ADC 16 bits, I2C, 100 kHz)

- Bus I2C 0: @0x48 (FSR 1–4), @0x49 (FSR 5–8) — junto a la IMU
- Bus I2C 1: @0x48 (FSR 9–12), @0x49 (4 × NTC)

ESP32-S3 — ESP-IDF + FreeRTOS (sensor_manager)

- adc_task — lee los 4 ADS1115 por I2C
- imu_task — lee LSM9DS1, notifica por evento (~50 Hz) vía xTaskNotifyGive()
- console_task — logging opcional (Kconfig CONFIG_DAS_ENABLE_CONSOLE_TASK)
- Escalado dinámico de frecuencia CPU + tickless idle entre lecturas

packet_builder — formato RAW v2

- 25 × int16 big-endian (9 IMU + 12 FSR + 4 NTC)
- Header 8B (tipo+versión+seq+ts_ms) + payload 50B + CRC-8 = 59 B

ble_transport — servidor GATT (NimBLE)

- CHR_SENSOR: notify del paquete de 59 B a ~30–50 Hz
- CHR_CMD: comandos OTA (start/chunk/end/abort) desde la app
- Intervalo de conexión negociado: 20–30 ms (ble_gap_update_params)

ota_manager

Actualización de firmware por BLE, sin flasheo físico

GATT notify

2. Aplicación móvil (Flutter)

ble_service.dart + foreground_service.dart

- Cliente BLE central; servicio en primer plano Android /
- modo bluetooth-central en iOS; reconexión automática 2-5-10 s

packet_decoder.dart

- Verifica byte de versión de protocolo y CRC-8
- Descarta (con log) cualquier paquete inconsistente

sensor_calibration.dart

- RAW → g, °/s, µT, gramos (curva FSR), °C (Steinhart-Hart)
- Único fichero a tocar para recalibrar (sin reflashear)

sensor_provider.dart (Riverpod)

- Orquestador central: detección de pasos (delta relativo),
- publica a MQTT, notifica UI e historial

UI (features/)

- dashboard: mapa de calor,
- mapa de presión, horizonte
- history / debug / profile

history_service.dart

- SQLite (sqflite) embebida
- pasos/día + step_events
- (trazabilidad FSR/NTC)

mqtt_service.dart

- mqtt_client sobre WebSocket seguro (wss), reconexión automática
- Topic: flutter_pisada/<id_dispositivo>/telemetry (o /test en modo simulado)

MQTT / WSS :8084

features/ota

Envío del firmware por BLE (CHR_CMD) desde la app

3. Plataforma en la nube (universidad)

Broker MQTT

- broker.emqx.io — WebSocket seguro, puerto 8084
- Suscripción a flutter_pisada/#

Servicio de ingesta (plataforma universitaria)

- Se suscribe al broker y decodifica el JSON de cada mensaje
- Inserta cada lectura como fila en la base de datos

PostgreSQL

- Almacenamiento persistente a largo plazo
- Serie temporal de presión, temperatura e IMU

Grafana

- Panel de visualización histórica
- Consultas SQL directas sobre PostgreSQL

Nota

- Esta capa es complementaria: la app conserva un
- historial local (SQLite) y funciona sin depender de ella.